SecDevQA: Can AI Agents Answer Developers' Security Queries During Bug Fixing Tasks?

Anonymous Authors

Abstract*—Developers routinely raise security queries in open-source project forums—whether a disclosed vulnerability affects their specific usage, which release fixes it, or whether a proposed patch is sufficient—sometimes as an explicit question, sometimes as a vulnerability report whose resolution they need. Answering such queries often requires cross-referencing artifacts that live partly outside the codebase—advisories, fix commits, dependency manifests—yet no existing benchmark evaluates LLMs or coding agents on the security queries developers actually raise, nor on the artifact dependence those queries exhibit. We present SecDevQA, a benchmark of [N] developer security query–answer pairs mined from issue threads across [K] open-source repositories. Each pair is synthesized into a self-contained, resolution-free query with a thread-grounded gold answer, and labeled along two axes: its *knowledge type*—whether a correct answer follows from general security knowledge (parametric) or requires project- or advisory-specific information (grounded)—and how the maintainer answered—by citing an *external reference* (a CVE/GHSA identifier, a fixed version, a fix commit or pull request, or a documentation or issue link) or from explanation alone. We also annotate the artifact types the maintainer drew on. Systems are evaluated at the moment the query was raised: an agent receives tool access over a *time-capped snapshot* of the project (repository at the last pre-report commit; issues, pull requests, and advisories published before the report), so that answering is reconstruction of the maintainer's reasoning rather than retrieval of the posted resolution. Grading is two-tiered and condition-aware: a deterministic identifier check scores each verifiable fact only under conditions where it is knowable, and a per-claim LLM judge—validated against human labels—scores the free-text remainder, with fabricated specifics reported separately as hallucinations. Analysis of the corpus yields a taxonomy of developer security query categories and an empirical map from categories to required artifact types; selectively granting and withholding artifact access to LLMs, and coding agents quantifies the marginal value of each artifact type per category.

***Index Terms*—software security, developer questions, benchmark, large language models, coding agents, question answering, mining software repositories**

I. Introduction

Developers regularly raise and resolve security queries in the course of building and maintaining software—sometimes as an explicit question, sometimes as a vulnerability or bug report whose resolution they need. They need to know whether a freshly disclosed vulnerability in a dependency actually affects the way they use it, security best practices, which release first introduced a regression, whether a proposed patch is sufficient, or which version is safe to upgrade to. When the answer is not obvious, they ask—on issue trackers, in advisory discussions, and on security related threads of the projects they depend on. How developers resolve such queries has direct consequences for the security of the software they ship: a substantial body of usable-security research shows that developers lean heavily on online information sources when solving security problems, and that the quality of those sources measurably shapes the security of the resulting code. Acar et al. [1] found that only a small fraction of the Stack Overflow threads developers consult for security tasks contain secure advice, and Fischer et al. traced insecure snippets from such threads into hundreds of thousands of shipped applications [2]. Getting trustworthy answers to developer security queries is therefore not a convenience problem but a software-security problem.

These queries are also distinctive in what it takes to answer them. Unlike general code-comprehension questions, a developer security query typically cannot be resolved from the repository's source alone. Answering "does CVE-2022-25883 affect me, how to make my code more secured, and what do I upgrade to? **[Put real issue references]**" requires cross-referencing artifacts that live partly outside the codebase: the CVE or GHSA advisory and its affected-version ranges, the fix commit or pull request, the project's dependency manifest, and sometimes documentations. Project maintainers answer these queries precisely because they know which artifact to consult—and, crucially, *different queries demand different artifacts*. A query about a transitive dependency is answered from the advisory and the manifest; a query about a TLS validation change is answered from the source and the commit history. Which artifact types are necessary for which kinds of security query is, to date, an unmeasured empirical question.

Large language models (LLMs) and coding agents are increasingly the first responders to such queries, yet no existing benchmark evaluates them on the security queries developers actually raise, or on the artifact dependence those queries exhibit on real software repositories. Repository-level QA benchmarks—SWE-QA [3], CoReQA [4], and StackRepoQA [5]—treat developer questions uniformly and consider only code-internal context; they neither isolate security queries nor model the external advisory artifacts those queries depend on. (StackRepoQA further reports that much of the apparent accuracy on such tasks reflects memorization of public threads rather than reasoning [5], a contamination risk any security-QA benchmark must confront.) Security-specific LLM benchmarks, in turn, measure a different task: SecVulEval [6] and CodeSecEval [7] evaluate vulnerability detection and secure code generation, SecureAgentBench [8] evaluates whether agents write secure patches, and SecQA [9] tests textbook security knowledge via multiple-choice questions. None of them studies real developer security queries, and none treats

artifact context as a controlled variable.

We introduce **SecDevQA**, a benchmark of real developer security *queries* paired with the maintainer answers that resolve them, mined from the issue and discussion threads of open-source GitHub repositories. We use *security query* to denote a developer's security information need as it surfaces in a project thread—whether posed as an explicit question (e.g., "does this CVE affect my usage, and which release fixes it?") or raised as a vulnerability or bug report whose resolution the developer needs—and §II-D describes how each thread is synthesized into a self-contained query–answer pair. Each pair is annotated so that a free-text security answer can be graded fairly. A *knowledge-type* label marks the pair as *parametric* (answerable from general security knowledge or a public standard) or *contextual* (resting on the project's own behavior or releases); this keeps the no-context condition an honest capability test rather than a memorization probe, and makes high no-context accuracy on contextual pairs a direct signal of contamination. Beyond knowledge type, we record whether the answer cites a concrete *external reference*—a CVE/GHSA/CWE identifier, a fixed version, a fix commit or pull request, or an issue, advisory, or documentation link. Not every security query is resolved by pointing to such a reference: many are, but others are answered correctly from explanation alone, so we treat reference use as an observed property of the answer rather than an inclusion requirement. Where a reference is a checkable identifier we additionally capture it as a *verifiable fact*, so that grading can first settle the objective question—does the response convey the named fact?—before any judgment of the surrounding prose.

SecDevQA also treats the evidence available to a system as a controlled variable. We annotate the artifact types each answer drew on, so each type can be granted or withheld rather than left as an implicit confound, and we evaluate systems as of the moment the query was raised. Each system runs over a snapshot of the project frozen at the last commit before the report, with the issue tracker, pull requests, and advisory database restricted to entries published by then; the posted resolution, which usually comes later, is therefore unavailable by construction. The task is to reconstruct the maintainer's reasoning from the evidence then available, not to retrieve an answer already written down. When a fix or duplicate report *did* precede the query, locating it in the frozen corpus is itself the realistic task a maintainer faces.

This design lets us pursue four questions prior work cannot. First, by inductively open-coding the mined corpus, we derive a taxonomy of developer security query categories and an empirical map of which artifact types maintainers draw on to answer each; these findings about developer security practice stand on their own, independent of any model evaluation (RQ1). Second, we measure how accurately frontier LLMs, open-weight LLMs, and coding agents answer these queries, broken down by category and knowledge type (RQ2). Third, we ask which artifact types are *necessary* for which categories, selectively granting and withholding each within the time-capped snapshot and measuring its marginal contribution (RQ3). Fourth, because our agent's tools are partitioned by artifact type, its transcripts reveal which artifacts it consulted, letting us separate retrieval failures, where the agent never reaches the right artifact, from reasoning failures, where it reaches the artifact yet still answers incorrectly (RQ4). Our guiding hypothesis is that context dependence is category-specific: an artifact type essential for one category of query yields little benefit for another, with direct implications for how retrieval-augmented and agentic security tools should be built.

Contributions. This paper makes three contributions:

- **SecDevQA**, a benchmark of real developer security query–answer pairs synthesized into self-contained, resolution-free evaluation items (verbatim originals retained) and annotated with knowledge type, required artifact types, whether the answer cites an external reference, and a structured record of any verifiable facts it states. We release it together with the mining and extraction pipeline and the evaluation harness (§II).
- A **time-capped evaluation methodology** that freezes the repository, issue tracker, and advisory database at the moment each question was asked, with a condition- and knowledge-type-aware two-tier grading protocol whose LLM-judge component is narrowly scoped and human-checkable (§III).
- An **empirical study of developer security queries**: an inductively derived taxonomy of query categories and a category-to-artifact map (both independent of any model evaluation), together with an evaluation of frontier LLMs, open-weight LLMs, and coding agents (a typed-tool reference agent and off-the-shelf agents) under selective artifact provision, quantifying each artifact type's marginal, category-specific contribution and separating agents' retrieval failures from reasoning failures (§II-D3, §III).

II. Benchmark Construction

[The numeric data throughout the paper will be iteratively updated, hence they are colored and bold]

SecDevQA is built in three phases. **(1) Query sourcing** (§II-B) selects source repositories from the GitHub Advisory Database and mines their issue and discussion threads. **(2) QA pair extraction and verification** (§II-C) runs a two-stage LLM pipeline that detects and extracts candidate security query–answer (Q&A) pairs—together with the artifact types and externally verifiable facts each answer draws on—after which the authors manually verify every candidate for inclusion. **(3) Synthesis and coding** (§II-D) turns each verified pair into one or more self-contained, gradeable evaluation items—each carrying a dual-verified per-item grading rubric (§II-D2)—under independent dual review, and inductively codes the pairs—open coding followed by higher-level axial coding—toward a taxonomy of developer security queries.

A. Security Query

We adopt the standard security construct from dependable computing: the preservation of **confidentiality**, **integrity**, and

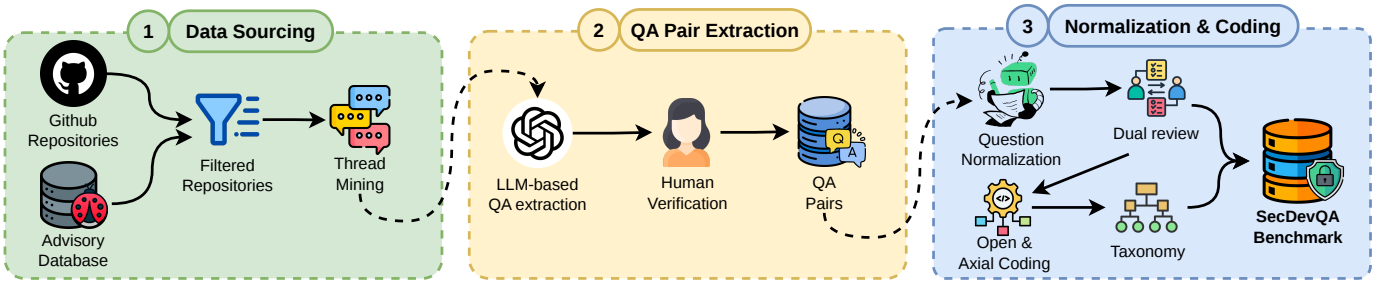

Fig. 1. The SecDevQA benchmark construction pipeline.

availability (CIA) against a deliberate *adversary* [10]. A thread is a *security query* if and only if the concern it raises is a potential compromise of one of these properties by an adversary crossing a *trust boundary* [11], that is, whether some party could gain unauthorized access, escalate privilege, bypass a control, tamper with state, deny service to others, or expose private data. This adversarial criterion alone governs inclusion; a cited CVE, GHSA, or advisory serves as corroborating evidence and is not a prerequisite. The decisive test is reachability across a trust boundary: a crash triggered by attacker-controlled input, such as a service parsing an untrusted upload, constitutes a denial-of-service compromise and is in scope. Scope depends on the concern a query raises and not on its eventual outcome, hence, a query asking whether a disclosed vulnerability affects the project remains in scope when the resolution is a not-affected determination, itself a security-triage need.

B. Query Sourcing

1) Repository Selection: To avoid hand-picking projects in a way that could bias the query distribution, we draw the source repositories from an external, independent maintained catalogue of security activity: the GitHub Advisory Database. We clone the database (snapshot **2026-06-04**; **31,213** reviewed advisories) and parse every reviewed GHSA entry, extracting the GitHub repositories referenced by each advisory and, for each repository, the number of advisories that carry a documented fix version. This yields **9,102** distinct repositories that have appeared in at least one reviewed advisory. We then apply three pre-specified filters intended to keep repositories that are both security-active and practically minable:

1) **Security activity:** at least **10** advisories with a documented fix version, so that the project has a substantive, externally catalogued security history;
2) **Popularity:** at least **1,000** GitHub stars, as a proxy for a real user base whose developers raise security queries;
3) **Maintenance:** a push within one week (as of **06-06-2026**) and not archived, so that threads reflect current practice.

Applying the advisory-count filter leaves **440** repositories; adding the popularity and maintenance filters leaves **344**. To enforce language and ecosystem diversity rather than concentrating on whichever ecosystem happens to have the most advisories, we rank the survivors by fix-bearing advisory count within each package ecosystem (npm, PyPI, RubyGems, etc.) and retain the top repositories per ecosystem, producing a diverse candidate pool spanning **ten** ecosystems.

From this pool we are mining repositories incrementally. The current corpus comprises the **ten** repositories in Table I, spanning **three** languages (Python, JavaScript, Ruby) and a range of security-sensitive domains: HTTP clients (`requests`, `urllib3`, `axios`), cryptography (`pyca/cryptography`), image processing (`Pillow`), web frameworks (`express`, `fastapi`, `rails`), authentication (`node-jsonwebtoken`), and payment integration (`stripe-node`). The selection deliberately mixes repositories with a large catalogued advisory history with widely used libraries in domains where security queries are common but formal advisories are sparser, so that the corpus is not skewed toward projects with heavyweight advisory processes (§II-C2 describes the per-repository cap that enforces this).

2) Thread Mining: For each repository we retrieve the issues since **2024-01 to 2026-05** and discussion threads and flatten them into ordered comment sequences. We bound the window deliberately: our goal is not to harvest every security query–answer pair a project has ever produced, but to collect a corpus large and diverse enough to support the taxonomy and evaluation in this study. Recent threads also better reflect current libraries, advisories, and developer practice. Additionally, we only considered the closed issues as they are more likely to contain resolved queries.

C. QA Pair Extraction and Verification

1) LLM Detection and Extraction: Security query–answer pairs are a sparse minority scattered across thousands of mostly-functional issues, and confirming one requires reading a thread end to end, so manual triage across tens of thousands of threads is impractical. We instead use a two-stage LLM pipeline as a *prefilter* that reads every thread and surfaces likely candidates, turning intractable full-text triage into tractable verification over a small candidate set.

Stage 1 (detection). An LLM (GPT-5.4-mini) reads each thread and decides whether it contains a genuine developer security query—an explicit question, or a security concern raised in a vulnerability or bug report paired with a substantive answer. Because this stage is a prefilter feeding human verification, we deliberately prompt it to favor *recall* over precision

(**Prompt Reference**): the cost of a false positive is a few seconds of reviewer time at the verification stage, whereas a false negative silently drops a real pair from the corpus with no opportunity for recovery **Need to set a validation report**. We accordingly set a permissive **0.70** confidence threshold and instruct the model to surface borderline threads rather than discard them. The downstream **39%** rejection rate (§II-C2) is the expected, acceptable price of this recall-oriented design.

Stage 2 (extraction). For threads passing Stage 1, a second prompt extracts the specific query and answer comments, the role of the answerer (maintainer, contributor, commenter, or self-answer), the artifact types the answer draws on (§II-C3), and any externally *verifiable facts* the answer states—CVE/GHSA/OSV identifiers, fixed version numbers, fix commit SHAs, fix PR references, and advisory URLs.

This pipeline produced **XXXXX** candidate pairs across the **ten** repositories. We intentionally extract candidates broadly at this stage and rely on manual verification, rather than the LLM, to make the final inclusion decision.

2) Manual Verification: Every candidate pair is reviewed by both authors through a purpose-built annotation interface that presents the full thread alongside the extracted query, answer, role, artifacts, and verifiable facts. A pair is *accepted* only if it satisfies the inclusion criteria: the query is a security query in the sense of §II-A—its resolution turns on a potential compromise of confidentiality, integrity, or availability by an adversary crossing a trust boundary—the answer is substantive and information-providing, and the query and answer are attributable to distinct, identifiable comments. Pairs whose faults involve no adversary and no trust boundary (e.g. ordinary functional or robustness bugs), that are unanswerable without information absent from the thread, or that are mis-extracted are *rejected*; the reviewer records a short reason for each rejection.

To prevent the most security-active projects from dominating the corpus, we cap the number of verified pairs retained per repository at **25**. This keeps the corpus balanced across projects, languages, and security domains rather than skewed toward the few repositories with the densest advisory history. In the current corpus the cap does not yet bind—every repository falls at or below it (the largest, `requests`, contributes **24** pairs)—but it fixes the composition policy as mining scales to more and larger repositories.

Of the **XXXXX** candidates, **158** (**60.8%**) were accepted and **102** were rejected—a rejection rate that, given the recall-oriented prefilter, is expected and underscores why LLM extraction alone is insufficient and human verification is required. **Report inter reviewer agreement** No single repository exceeds **15.2%** of the verified pairs. The accepted pairs are answered predominantly by project maintainers (**104** of **158**), with the remainder from contributors (**26**), other commenters (**18**), and the original poster self-answering (**10**). Threads span **2020–2026** and contain a median of **4** comments (mean **5.7**, max **41**).

Inclusion does not require an answer to point to an artifact: it turns only on the criteria above. Instead, we *record* whether each answer cites an *external reference*—a CVE/GHSA/CWE/OSV identifier, a fixed version, a fix commit or PR, or an issue, advisory, documentation, or other external link—and treat this as an observed property rather than a filter. The answer cites at least one such reference in **80%** of verified pairs; the remaining **20%** resolve the query from explanation alone and are graded by the judge, and we retain them in full so the taxonomy is not biased toward only those queries that resolve to a citable artifact—some security queries are answered correctly without one, and that is itself a finding. The subset of references that are checkable identifiers additionally anchors deterministic grading; the most common are fixed version numbers (**47** pairs) and advisory URLs (**44**), followed by fix PRs (**26**), CVE identifiers (**24**), fix commits (**21**), and GHSA identifiers (**14**). These identifiers are spot-checked against NVD, GHSA, and OSV at construction time, and the advisory-database snapshot date is recorded with the release to support future re-verification.

3) Artifact-Type and Verifiable-Fact Annotation: Each verified pair carries an `artifacts_needed` list recording which artifact types the maintainer's answer drew on, and a structured `hard_facts` record. The artifact-type distribution (Table II) shows that answering these queries is rarely a matter of reading source code alone: documentation and project source dominate, but advisories, pull-request and commit history, dependency manifests, and external references (RFCs, registries, upstream issues) each appear in a non-trivial fraction of answers. This spread is what motivates the per-artifact context conditions in the evaluation (§III): different query categories lean on different artifact types, and the annotation makes that dependency measurable.

D. QA Synthesis and Coding

1) QA Synthesis: A verified thread is not yet a gradeable item: the query is embedded in a multi-comment exchange, leans on earlier comments ("the error above"), and often sits in a thread whose later comments—or title—already disclose the resolution. Handing such a thread to a model under test would leak the answer and conflate reading comprehension with security reasoning. We therefore synthesize each verified pair into one or more self-contained *(query, answer)* items with a single `GPT-5.5` pass that is then human-verified, under one governing constraint: each item must stay faithful to what was really raised and answered. Because a security query surfaces as often as a vulnerability or bug report as it does as an explicit question, synthesis renders the developer's information need as a self-contained query regardless of its original surface form, without importing any detail the thread does not contain. Each item carries the synthesized query and thread-grounded gold answer, the report time t_θ (the issue's creation time), the knowledge-type label below, the artifact types the answer drew on (§II-C3), and the verifiable-fact record—partitioned into *subject* facts (CVE/GHSA/CWE/OSV identifiers a reporter cites as a premise) and *fix* facts (fixed versions, fix PRs/commits, advisory URLs that constitute the resolution). Figure 2 contrasts a raw mined thread with the item it

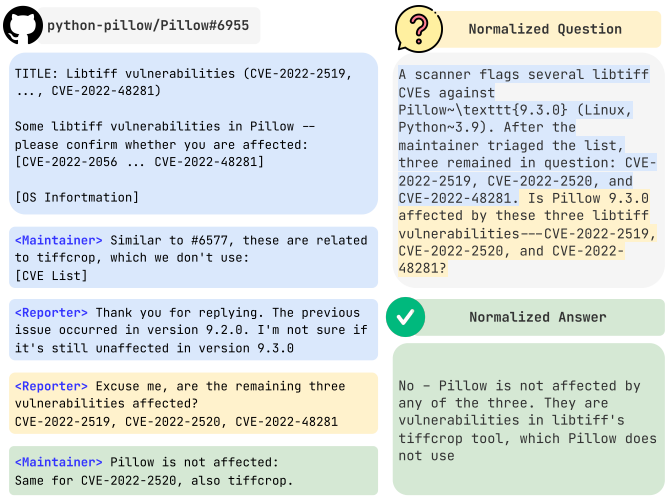

Fig. 2. QA synthesis turns a verified thread into a gradeable item (`python-pillow/Pillow` #6955; §II-D1). Color marks the three roles in both panels: premises / context, the question / problem statement, and the answer.

becomes, and makes the transformation concrete; we describe its components next.

Self-contained, resolution-free queries. The query preserves every reporter-supplied specific verbatim (versions, OS/runtime, configuration, error and stack-trace text, code and proof-of-concept) and resolves dangling references against earlier comments, but never states its own resolution: a fix fact introduced *as the answer* is withheld, so the item stays an open query rather than a fill-in-the-blank. To make this auditable we retain the verbatim original comment and run a deterministic leak check that flags any fix fact appearing in the query without having been stated by a participant *before* the maintainer's answer—such an identifier can only have been imported from the answer, whereas a fix the reporter cited earlier is legitimate context. Authors reviewed the flagged items and found them to be predominantly false positives. Flagged items are corrected in the human pass, and we report the residual leak rate.

Thread-grounded answers. The gold answer faithfully distills how the maintainer answered, drawn strictly from the thread; the synthesis must not add facts, fixes, or reasoning the thread lacks. A terse pointer—"fixed in #932, released 9.0.2"—is kept as exactly that. We do not fetch external advisories or patch diffs to enrich the gold answer, which would fabricate ground truth the thread never contained; instead the artifacts it points to are recorded as `grounding_sources` and the referenced diffs are resolved at grading time under the with-context conditions (§III), so a terse pointer can still corroborate a richer but correct model response.

Knowledge type. Each item is also labelled by *knowledge type*, recording whether answering it requires this project at all. A *parametric* item is answerable from general security knowledge or a public standard the query cites. In `fastapi/fastapi` #5837,[1] a login cookie never reaches the browser, and the answer is a browser-platform rule rather than a fact about FastAPI: a `SameSite=None` cookie must also set `Secure`. A *contextual* item instead rests on this project's own behavior, releases, or a source the thread points to. In `auth0/node-jsonwebtoken` #921,[2] the answer "the fix was merged in #932 and released in 9.0.2" cannot be produced without the repository. A deterministic rule hardens the label: an answer stating a fix fact about the project's own code or releases is forced *contextual*, so a borderline judgment never admits an unanswerable pair into the no-context test. The label exists for the evaluation, where it keeps the no-context condition a fair capability test and serves as a contamination probe (§III).

Outcome. An initial pass over **50** threads yields **56** items (**3** threads decomposed), **44** contextual and **12** parametric. Contextual pairs predominate, since an answer that cites the project's own fix is contextual by construction, but the **21%** parametric share (secure-configuration, secure-API-usage, advisory-triage, and design-rationale queries) keeps the no-context capability test and the contamination analysis well populated. Every item retains its verbatim original, and *two researchers independently review every pair*, letting us report Cohen's κ on the knowledge-type label and the leak and context-completeness judgments over the whole benchmark and reconcile all disagreements before evaluation.

2) Per-Item Grading Rubrics: Model answers to these queries are open-ended free text. An n-gram overlap with the gold answer is therefore an inadequate measure of correctness, and a single holistic quality score from an LLM judge is both noisy and difficult to audit [12], [13]. We adopt *rubric-based* evaluation, which decomposes each judgment into atomic, separately verifiable criteria and has become established practice for making LLM-as-judge scoring more reliable and interpretable [14], [15]. Because every security query has its own notion of a correct answer, one item turning on a specific cookie attribute and another on a particular fixed release, the criteria must be *instance-specific* rather than drawn from a fixed global scale [16]. We accordingly author one rubric per evaluation item during construction, following the use of expert-written, per-example rubrics to anchor evaluation in other high-stakes grounded domains [17].

Criteria. Each criterion carries an *axis*. *Correctness* criteria state the load-bearing facts or verdicts that must hold, such as whether the project is affected, the fixed version, or the root cause the answer establishes. *Completeness* criteria state the supporting coverage a thorough answer adds, such as the underlying mechanism, a scope caveat, or an alternative valid fix. The axis is a kind rather than a numeric weight: correctness *gates* the item outcome while completeness only contributes coverage, so a missed load-bearing fact cannot be offset by surrounding detail, reflecting that incorrect security guidance is unsafe however thorough it is. Figure 3

[1]https://github.com/fastapi/fastapi/issues/5837
[2]https://github.com/auth0/node-jsonwebtoken/issues/921

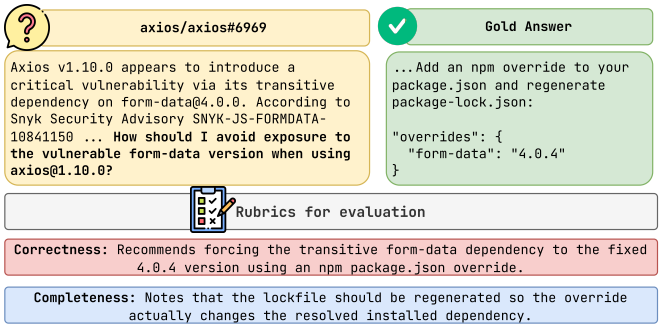

Fig. 3. An example rubric for a question answer pair.

shows an example of the generated rubrics across two axes for a transitive dependency-related query in `axios/axios` repository. Every criterion must be supported by a verbatim span of *answer-side* evidence, namely the gold answer, the maintainer's reply, a referenced fix diff, an advisory, or a recorded verifiable fact, and a criterion grounded only in the query (which the candidate already receives) is discarded so that the rubric cannot reward a model for restating the question. We keep each rubric short, at most **five** criteria, so that a long checklist of easy criteria does not saturate and flatten the differences between systems.

Resolution timing. Criteria are graded as facts a competent developer could supply rather than strings to reproduce, and each rubric is keyed to when the fix existed relative to the report time t_θ (§II-D1) so that it matches how the maintainer answered. When the fix predated t_θ it was already retrievable from the project, and the expected contribution, as it was for the maintainer, is to *identify* it. The rubric then credits naming the fix (a pull request, commit, or release) and records equivalent acceptable forms, rather than requiring that the patch be reproduced. When the fix landed only afterward it was not retrievable at query time, so the rubric instead asks whether a proposed remediation moves in the *same direction* as the eventual fix, and it marks that criterion unknowable at t_θ so that it is never scored under conditions that lacked the artifact. For `node-jsonwebtoken` #921, whose maintainer answer points to an upstream fix, the resulting rubric pairs a correctness criterion (establish that the issue is fixed and name pull request #932 or release 9.0.2) with a completeness criterion naming the weakness it addresses. A response that conveys the resolution by another route satisfies the former through its recorded alternatives, whereas one that merely echoes the reporter's symptom satisfies neither.

Verification. We draft each rubric with `GPT-5.5`, prompting it with the synthesized query, the thread-grounded gold answer, and every artifact the answer draws on, namely the maintainer's reply, any referenced fix diff, the advisory, and the verifiable-fact record, subject to the axis, grounding, and length constraints above. Two researchers then independently verify every rubric criterion, editing or removing any criterion not grounded in the answer-side evidence, and we reconcile all disagreements before grading. We report inter-rater agreement on the criteria as Cohen's $\kappa = \mathbf{0.00}$. A rubric is frozen once both reviewers accept it, and the rare item whose fix diff is too large to ground a criterion automatically is flagged for manual authoring rather than dropped.

3) Open and Axial Coding: We code the verified pairs in two stages: inductive *open coding* of each pair, followed by higher-level *axial coding* that collapses the open codes into a smaller set of categories. Open coding characterizes *what* developers raise; the coding unit is the verbatim query–answer exchange. An LLM coder (`gpt-5.4-mini`, accessed through a uniform LiteLLM interface) is prompted to assign one or two short noun-phrase codes naming the security concern at issue, given the issue title and body, the query–answer summary, the full thread, and the focal query and answer. As with detection, an LLM-assisted first pass makes coding tractable and consistent at corpus scale while keeping a human in control of the final code: the first author then reviewed every pair in the same annotation interface, free to edit or replace any LLM-assigned code, so the resulting codebook reflects human judgment rather than model output. The first-author verification note is deliberately withheld from the coder to avoid anchoring its codes to the acceptance rationale.

This pass produced **261** code instances over the **158** pairs (**1–2** per pair), with **235** distinct codes—a fine-grained vocabulary that reflects how specific developer security queries are (e.g. "missing subject alternative names," "HTTPS redirect scheme downgrade," "transitive dependency vulnerability"). The single most frequent code, *transitive dependency vulnerability*, accounts for **12** instances; the long tail of singleton codes confirms that the corpus is not dominated by a handful of canned queries.

Axial coding then groups the open codes into a smaller set of higher-level categories that anchor the per-category analysis in RQ2–RQ3. The grouping is emergent—codes are merged by conceptual affinity rather than fitted to a predefined list—but each resulting category is named and delimited by the security concern it captures: the property placed at risk (confidentiality, integrity, or availability) and the developer's underlying intent, consistent with the construct of §II-A. This keeps the categories grounded in security semantics rather than surface phrasing, so the taxonomy reflects distinct classes of security query rather than an ad hoc partition of keywords. Table III presents a *preliminary, author-proposed* grouping of the **235** codes into **eleven** candidate categories, reported here to convey the shape of the emerging taxonomy. The two largest groups—HTTP request and response security (SSRF, redirects, headers, cookies) and TLS and certificate validation—together cover roughly half the corpus, consistent with the HTTP-client and web-framework projects mined so far. We stress that this grouping is not final: category names and boundaries will be fixed through axial coding with an independent second coder, and we will report inter-rater agreement (Cohen's κ) on a held-out sample, together with a saturation analysis as the corpus grows.

TABLE I

REPOSITORIES IN THE CURRENT SECDEVQA CORPUS. **PAIRS** = Q&A PAIRS RETAINED AFTER MANUAL VERIFICATION; **HARD** = SUBSET WHOSE ANSWER CARRIES AT LEAST ONE EXTERNALLY VERIFIABLE FACT (CVE/GHSA ID, FIXED VERSION, FIX COMMIT/PR, ADVISORY URL). STARS ROUNDED TO THOUSANDS (**2026-06-08**).

Repository	Lang.	Stars	Pairs	Hard
psf/requests	Python	54.0k	24	14
axios/axios	JS	109k	23	19
expressjs/express	JS	69.1k	23	17
pyca/cryptography	Python	7.6k	21	13
urllib3/urllib3	Python	4.0k	14	6
stripe/stripe-node	JS	4.4k	13	5
python-pillow/Pillow	Python	13.6k	12	12
rails/rails	Ruby	58.5k	11	9
auth0/node-jsonwebtoken	JS	18.2k	9	5
fastapi/fastapi	Python	99.0k	8	5
Total (10 repos)			158	105

TABLE II

ARTIFACT TYPES THE MAINTAINER'S ANSWER DREW ON, ACROSS THE 158 VERIFIED PAIRS (A PAIR MAY DRAW ON SEVERAL).

Artifact type	Pairs	%
documentation (docs, README, changelog)	126	79.7
code (repository source)	92	58.2
advisory (GHSA / OSV / project advisory)	35	22.2
pr_data (PR diff or discussion)	23	14.6
issue_tracker (linked issues)	23	14.6
dependency_manifest	19	12.0
external_reference (RFC, registry, etc.)	14	8.9
commit_history (log, blame, commit)	14	8.9
cve_cwe_db (NVD/CWE/CVSS)	7	4.4
ci_logs	2	1.3

III. EXPERIMENTAL DESIGN

A. Research Questions

RQ1: What categories of security concerns motivate developer questions in open-source projects, and which artifact types do maintainers draw on to answer them?

RQ2: How accurately do frontier LLMs, open-weight LLMs, and coding agents answer real developer security questions, broken down by question category and knowledge type?

RQ3: Which artifact types are necessary to answer which categories of developer security questions? What is the marginal accuracy contribution of each artifact type when access to it is selectively granted or withheld?

RQ4: When agents choose autonomously which artifacts to consult, do they consult the artifact types maintainers actually needed—and when they fail, is it because they never retrieved the right artifact or because they reasoned incorrectly over it?

RQ1 is answered by the benchmark construction itself (§II-D3); the category-to-artifact map it produces is simultaneously an empirical finding about developer security practice and the ground-truth prediction that RQ3 tests: if maintainers needed a given artifact type to answer questions of a category, then granting a system access to that artifact type should improve its accuracy on the category, and withholding it should specifically degrade it.

B. Evaluation at the Moment of Asking: Time-Capped Snapshots

A naive with-context evaluation would hand the system the repository at its current head—but for most items the fix commit, the fix pull-request discussion, and often the advisory itself entered the repository *after* the question was asked. Against current state, "answering" degenerates into retrieving the posted resolution. We instead freeze the world at the moment the question was asked. For an item with report time t_θ, the *time-capped snapshot* comprises: the repository working tree at the last default-branch commit preceding t_θ, with commit history truncated there; the project's issues and pull requests created at or before t_θ, with later comments and reviews removed; and the GitHub security advisories referencing the repository published at or before t_θ. The source thread itself is always excluded from the issue corpus—the system must never read the conversation it is being tested on. No live network access is provided under any condition: a system that consulted today's NVD entry would observe the resolution and void the cap.

Two properties of this construction deserve emphasis. First, resolution identifiers that post-date t_θ are unavailable *by construction*, which changes what can be graded (§III-E): for such items the task is to produce the correct diagnosis, verdict, and remediation direction, not to name a future release number. Second, when the resolution *pre*-dates the question—a duplicate report already answered, a fix already merged or released, an advisory already published—it is legitimately discoverable inside the snapshot, and locating it is precisely the task a maintainer performs when triaging a new report. The benchmark therefore contains a natural retrieval sub-task whose ground truth we did not have to synthesize.

C. Systems Under Test

We evaluate three system classes. Each agent backbone is also evaluated as a bare LLM, so every with-context result has a paired no-context control from the same model.

Bare LLMs (no context). The model receives only the synthesized question q and answers from parametric knowledge. We evaluate frontier closed-weight models and open-weight models under identical prompts (temperature 0 where the API permits).

Reference agent (typed tools). The reference agent interacts with the snapshot through a fixed set of read-only, typed tools. The tools are organized into five groups, one for each artifact type the snapshot exposes—CODE for the repository working tree, COMMITS for version history, ISSUES and PRS for the issue and pull-request corpora from GitHub, and ADVISORY for security advisories. Each group provides

TABLE III

[GIAS: NEED TO HAVE SOME HIGH LEVEL JUSTIFIABLE CATEGORIES] AXIAL GROUPING OF THE OPEN CODES INTO CANDIDATE HIGHER-LEVEL CATEGORIES. **PAIRS** COUNTS VERIFIED PAIRS TOUCHING THE CATEGORY (A TWO-CODE PAIR MAY FALL IN TWO CATEGORIES, SO THE COLUMN SUMS TO MORE THAN **158**).

Candidate category	Pairs	Representative open codes
HTTP Request and Response Security	**40**	raw request body, credential leakage on redirect, CSRF token generation, double submit cookie, automatic redirect handling, absolute URL bypass
TLS and Certificate Validation	**38**	SSL certificate verification failure, OpenSSL version mismatch, ASN.1 parsing error, TLS SNI hostname handling, certificate verification failure, stricter SSL verification
Dependency and Supply-Chain Security	**26**	transitive dependency vulnerability, transitive dependency CVE, compromised package versions, dependency version update, prepare script install failure, false positive dependency vulnerability
Authentication and Session Integrity	**17**	webhook signature verification, unsigned JWT handling, question regarding jwt signing, signature verification bypass, token blacklist, JWT logout invalidation
Cryptographic Key and Algorithm Use	**15**	SM2 key algorithm mismatch, RSA key length requirement, cryptographic algorithm default mismatch, post-quantum support roadmap, encrypted RSA key loading, silent hash digest change
Input Handling and Denial of Service	**15**	memory exhaustion DoS, unsafe boundary generation, ReDoS vulnerability, ReDoS protection, regex route matching, array index limit
False Positive Report Triage	**14**	false positive report, scanner false positive, false positive vulnerability report, false vulnerability report, vulnerability database error, advisory database update
Patch, Release, and Compliance	**14**	fixed version announcement, SSRF vulnerability backport, safe version guidance, patched version marking, TLS deprecation support, SSRF fix availability
Secure Configuration and Defaults	**13**	deprecated URL parsing, opt-in security setting, missing config types, android network security config, initializer order misconfiguration, missing rate limiting
Platform and Version Compatibility	**10**	RAT on machine, issue tracker reference, local version parsing, Windows Python compatibility, permission denied error, missing attribute regression
Concurrency and Data Integrity	**7**	global state mutation, thread safety data race, shared block cache, deadlocked transaction state, writes outside transaction, thread safety issue

a keyword-search operation and a retrieve-by-identifier operation, mirroring how a maintainer first locates a candidate artifact and then reads it in full, and every tool belongs to exactly one group; Table IV enumerates the resulting eleven tools.

Every tool draws exclusively from the time-capped snapshot (§III-B) and re-enforces the cap at the moment of access, such as a commit is shown only if its revision is an ancestor of the snapshot commit, the source thread is withheld from the issue corpus, and issue, pull-request, and advisory queries return only records within the snapshot window. Advisory records carry their aliased CVE and CWE identifiers inline, so vulnerability-database lookups are served from within the snapshot and no condition depends on live network access. The agent issues tool calls in a bounded loop—at most **X** rounds—before committing to a final answer, and we release every transcript.

Off-the-shelf coding agents. To ground the agent results in systems practitioners actually use, we additionally evaluate production coding agents (Claude Code and OpenCode) headlessly. Each item runs in a sandbox directory materializing its snapshot as plain files: the repository tree exported at the snapshot commit *without* version-control metadata (so post-report history is physically absent), the capped issue, pull-request, and advisory corpora as structured data files, and a correspondingly truncated commit log. Because a shell-equipped agent cannot be *provably* prevented from reaching the network, this condition is best-effort capped: the prompt forbids external access, the sandbox contains nothing post-report, and we release full transcripts; we treat the residual risk as a stated limitation (§IV) and rely on the typed-tool agent for the airtight analysis.

TABLE IV

TYPED TOOLS EXPOSED TO THE REFERENCE AGENT, GROUPED BY ARTIFACT TYPE (§III-C).

Tool	Returns (time-cap enforced)
CODE — working tree at the last commit $\leq t_\theta$	
`list_dir`	Directory listing within the snapshot tree.
`read_file`	Line-numbered file contents for a path and line range.
`search_code`	Regex search over tracked files (`git grep`).
COMMITS — history truncated at the snapshot commit	
`git_log`	Commit log, newest first, optionally path-scoped.
`git_show`	Message, stat, and patch for one SHA; rejected unless the SHA is an ancestor of the snapshot commit.
ISSUES — issues $\leq t_\theta$, source thread excluded	
`search_issues`	Keyword search over the capped issue corpus.
`get_issue`	Full issue thread by number, comments capped at t_θ.
PRS — pull requests $\leq t_\theta$	
`search_prs`	Keyword search over the capped pull-request corpus.
`get_pr`	Pull-request body and reviews by number, reviews capped at t_θ.
ADVISORY — GHSA/OSV records published $\leq t_\theta$	
`search_advisories`	Keyword search over advisories referencing the repository.
`get_advisory`	Full advisory by GHSA or CVE id, exposing aliased CVE and CWE identifiers and affected version ranges.

D. Context Conditions and Selective Artifact Provision

Let $\mathcal{G} = \{\mathsf{code}, \mathsf{commits}, \mathsf{issues}, \mathsf{prs}, \mathsf{advisory}\}$ be the tool groups. The annotated artifact types map onto these

groups in the obvious way: code, dependency manifests, and documentation are served by code; advisories and CVE/CWE database entries by advisory; external references have no serving group, since live external sources are excluded by the time cap. A *condition* is a subset $S \subseteq \mathcal{G}$ of groups the system may access:

- $S = \emptyset$ — **no context**: the bare-LLM condition;
- $S = \mathcal{G}$ — **full snapshot**: the agent conditions of §III-C, run over the entire benchmark;
- $S = \mathcal{G} \setminus \{g\}$ (leave-one-out) and $S = \{g\}$ (single-group) — **selective provision**, run on a stratified subset of contextual items whose annotated artifact types span at least two groups, so that ablating one group leaves the others intact.

The selective-provision runs operationalize RQ3 as a controlled experiment: necessity of group g for category k manifests as an accuracy drop under $\mathcal{G} \setminus \{g\}$ relative to $\mathcal{G}$, and sufficiency as accuracy under $\{g\}$ approaching that under $\mathcal{G}$ (§III-F). The full-snapshot agent runs simultaneously supply the RQ4 observational data: per item, the transcript yields the set of groups the agent actually consulted, which we compare against the groups serving the maintainer-annotated artifact types.

E. Grading

Model answers are free text, so grading combines a deterministic tier that is exact where exactness is possible with an LLM-judge tier that is validated where judgment is unavoidable. Both tiers are aware of the condition and the knowledge type.

Tier 1: condition-aware verifiable-fact matching. Each verifiable fact is matched in the response by a conservative, type-specific identifier pattern (word-boundary–guarded version strings, PR-shaped references, SHA prefixes of at least seven characters, exact CVE/GHSA/CWE identifiers). Crucially, a fact is *scored* only where it is knowable under the condition: subject facts the reporter could cite are gradeable under any condition, while a fix fact is gradeable only when the system had context access *and* the artifact behind the fact existed at report time (the fix commit merged, the release tagged, the advisory published before t_θ). A no-context model is therefore never penalized for not knowing one project's pull-request numbering, and no system is penalized for not naming an identifier that did not yet exist—the failure mode that, left uncorrected, turns a capability benchmark into a memorization probe. Unknowable facts are recorded as *not gradeable* and excluded from both numerator and denominator.

Tier 2: per-claim judge with a resolution oracle. An LLM judge—never the model under evaluation, with candidate identity anonymized—scores the response against the item's prepared rubric (§II-D2), verdicting each criterion (a cause, a verdict, a user-actionable remediation, an identifier) as met, partially met, or not met and quoting the supporting response phrase so every verdict is spot-checkable. Because gold answers are thread-grounded (§II-D1), a terse maintainer pointer ("fixed in #932, released 9.0.2") yields a terse gold; to let a richer-but-correct response earn credit such a gold cannot confirm, items whose gold answer cites a fix pull request or commit are enriched at grading time with the resolved change diff as a *resolution oracle* for the judge. Eligibility for this oracle is deliberately mechanical—an explicit fix PR or commit reference, resolved by a single API call, with failures marked unresolved and the item graded on the thread gold alone—and we report the eligibility and resolution rates rather than chasing fixes through advisory prose. The judge is condition-aware in one further respect: under agent conditions, specific detail beyond the terse gold (affected ranges, root-cause analysis, identifiers the agent verifiably retrieved) is expected enrichment, and is flagged as hallucination only when it *contradicts* the gold or the question's premises; under no context, any concrete project-specific assertion absent from the gold is flagged.

Outcomes. Each (item, system, condition) triple receives an outcome in $\{\text{CORRECT}, \text{PARTIAL}, \text{INCORRECT}\}$ by rolling the per-criterion verdicts up along the rubric axes (§II-D2): correctness *gates* the bucket—an item is CORRECT only when every correctness criterion is met, INCORRECT when any is missed or contradicted, and PARTIAL otherwise (correctness only partially met, or met with thin completeness coverage). Completeness criteria refine within the bucket and are reported separately as a coverage rate, but never override correctness, so we need no numeric criterion weights. Orthogonally, a boolean HALLUCINATED flags a response that asserts fabricated specifics. We keep hallucination separate rather than folding it into INCORRECT: an answer can convey the right remediation *and* fabricate a CVE number, and the fabrication is the more safety-relevant signal.

Judge reliability. We use an ensemble of ≥ 2 judge models with randomized presentation order; disagreements are flagged for human resolution. Judge verdicts are cross-checked against the Tier-1 identifier matches, and mismatches (e.g. a CORRECT verdict with zero gradeable facts matched) are audited. We calibrate against human labels on a stratified random sample of **50–80** graded items and report Cohen's κ (judge vs. human); the judge prompt is revised and re-calibrated until $\kappa \geq 0.80$ before any reported run.

F. Metrics

Our primary metric is $\mathrm{Acc}(S, k)$, a system's accuracy under condition S on the items of question category k, alongside the analogous partial-credit rate, the hallucination rate, and the Tier-1 verifiable-fact recall over gradeable facts. All metrics are additionally split by knowledge type: no-context capability claims are made on parametric items only, while contextual items carry the artifact-context analysis. The marginal value of artifact group g for category k is the leave-one-out contrast on the selective-provision subset, $\delta_g(k) = \mathrm{Acc}(\mathcal{G}, k) - \mathrm{Acc}(\mathcal{G} \setminus \{g\}, k)$, and its sufficiency is $\sigma_g(k) = \mathrm{Acc}(\{g\}, k) - \mathrm{Acc}(\emptyset, k)$, both over contextual items. RQ1's category-to-artifact map predicts $\delta_g(k)$ to be large exactly where maintainers needed an artifact type served by g; RQ3 tests that prediction. For RQ4 we report, per category,

the agreement between the groups the agent consulted and the groups serving the maintainer-annotated artifact types, and decompose agent failures into *retrieval failures* (the agent answered incorrectly having never consulted some required group) versus *reasoning failures* (it consulted every required group and still answered incorrectly).

G. Negative Controls and Contamination Analysis

All threads are public and plausibly present in training corpora; a credible security benchmark must measure, not assume away, memorization. We use three instruments. First, the knowledge-type design itself: contextual items are near-unanswerable without project access, so above-chance no-context accuracy on contextual items is direct evidence of memorization, cleaner than any post-hoc heuristic. Second, every item carries t_θ, and results are stratified by whether t_θ precedes each model's training cutoff; markedly higher no-context accuracy on pre-cutoff items indicates recall rather than reasoning. Third, for the typed-tool agent, the transcript makes memorization visible at item granularity: a response naming a fix identifier whose serving tool group was never called cannot have derived it from the provided context. Approximately **10%** of items are additionally tagged as unanswerable from their annotated artifact types alone (requiring, e.g., an external database lookup excluded by the time cap); these negative controls measure confabulation separately from legitimate inference.

IV. Threats to Validity

Construct validity. Our evaluation items are *synthesized* from raw threads—made self-contained and stripped of their resolution—which could dilute the authenticity claim or leak the answer. Both risks are addressed by construction (§II-D1): the verbatim original reporter text is released with every item; the deterministic leak check flags any fix identifier present in a question that the reporter had not stated before the answer, and we report the residual leak rate of the released set; and every synthesized item is independently reviewed by two researchers, with Cohen's κ reported over the full benchmark for the knowledge-type, leak, and context-completeness judgments and every disagreement reconciled before evaluation. The knowledge-type label itself is a construct: mislabelling a contextual pair as parametric would admit an unanswerable question into the no-context capability test. The deterministic hardening rule (§II-D1) biases errors in the safe direction, and condition-aware gradeability ensures a labelling error cannot convert an unknowable identifier into a scored miss. Finally, gold answers are thread-grounded: a terse maintainer pointer yields a terse gold. We do not inflate such golds; instead the grading-time resolution oracle (§III-E) supplies the resolved fix diff to the judge, and we report results separately for pointer-style and substantive-answer items so the two populations are not conflated.

Internal validity. Free-text grading uses an LLM judge, and verifiable facts ground but do not eliminate judge error—a response may cite the correct CVE while contradicting the verdict it implies. We mitigate with the controls of §III-E: per-claim verdicts with quoted evidence rather than holistic scores, a $\geq$ 2-judge ensemble with flagged disagreements, regex cross-checks of identifier verdicts, and human calibration to $\kappa \geq 0.80$ on a stratified sample before any reported run. The resolution oracle is scoped by a mechanical eligibility rule (an explicit fix PR or commit reference, one resolution attempt); items outside it are graded on the thread gold alone, and we report eligibility and resolution rates so the oracle's coverage is visible. Snapshot assembly is itself an instrument that can err: the snapshot commit is resolved on the default branch (a question may concern a release branch), and the issue and advisory corpora are bounded by what our miners retrieved. We release snapshot identifiers (commit SHAs, corpus counts, database snapshot dates) with every run for re-verification.

External validity. We mine only publicly visible artifacts. Security issues are often disclosed privately and resolved under embargo, leaving just an advisory or changelog in public; SecDevQA captures questions negotiated in the open and may under-represent privately routed ones, so our claims are scoped to publicly resolved security Q&A. Repositories are selected for catalogued security activity and popularity (**three** languages so far), which may bias toward projects with formal advisory processes; we report per-repository results and cap any repository's share of the corpus to keep the bias bounded and visible. The hard-verifiability filter excludes question types that rarely yield checkable identifiers (e.g. severity assessment, design rationale); deriving the taxonomy from the full corpus, including non-hard pairs, keeps that gap measurable rather than silent.

Contamination and the time cap. Public threads may be memorized [5], and the time cap constrains a system's *context*, not its *weights*: a model whose training data includes the resolved thread can reproduce the resolution under any condition. We therefore measure memorization rather than assume its absence, with three instruments (§III-G): no-context accuracy on contextual items, training-cutoff stratification over t_θ, and transcript-level detection for the typed-tool agent (a fix identifier asserted without the serving tool group ever being called). A residual limitation remains for off-the-shelf agents: a shell-equipped agent cannot be provably prevented from reaching the network, so the external-agent condition is best-effort capped—the sandbox physically contains no post-report data and the prompt forbids external access—and we release its transcripts; airtight claims rest on the typed-tool reference agent.

V. Related Work

Repository-level developer question answering. A recent line of benchmarks evaluates LLMs on questions about real codebases. SWE-QA [3] constructs repository-level questions from a taxonomy of seed templates instantiated per repository with program analysis and an LLM, pairing them with RAG-assisted, expert-revised answers and an LLM-as-judge evaluation with human validation. Its questions are clean and controllable but deliberately synthetic: no developer ever asked

them. CoReQA [4] similarly targets repository-level comprehension with code-centric context. StackRepoQA [5] moves toward authenticity by pairing Stack Overflow questions with repositories, and reports a finding that motivates much of our design: a substantial share of apparent model accuracy on public developer threads reflects memorization of those threads rather than reasoning over the repository. All three treat developer questions as a uniform population, supply only code-internal context, and grade against answers whose correctness is itself difficult to verify externally. SecDevQA differs on each axis: its questions are verbatim-grounded in real maintainer exchanges, are specifically security questions whose answers depend on artifacts *outside* the repository (advisories, vulnerability databases), carry externally verifiable facts that anchor grading, and are evaluated under a time cap that makes the memorization risk StackRepoQA observed measurable per item (§III-G) rather than merely acknowledged.

Security benchmarks for LLMs and agents. Security-specific evaluations target tasks adjacent to ours but distinct from it. SecVulEval [6] measures vulnerability *detection* on labelled code; CodeSecEval [7] measures secure code *generation*; SecureAgentBench [8] evaluates whether coding agents produce secure patches under repository tasks; and SecQA [9] tests textbook security knowledge through multiple-choice questions. These benchmarks share deterministic or near-deterministic grading, but none evaluates the question-answering task developers actually pose to maintainers—free-text, situation-specific, and resolved by consulting heterogeneous artifacts—and none treats artifact access as a controlled variable. SecDevQA is complementary: where SecureAgentBench asks whether an agent can *write* a secure change, we ask whether it can *answer* the security questions that precede and motivate such changes, and which evidence it needs to do so.

Developer security information behavior. That developers' security outcomes depend on the information sources they consult is well established. Acar et al. [1] showed experimentally that developers permitted to consult Stack Overflow produced less secure code than those using official documentation, and that only a minority of consulted threads contained secure advice; Fischer et al. [2] traced insecure code snippets from such threads into hundreds of thousands of deployed Android applications. This literature measures the *consequences* of answer quality but offers no instrument for evaluating a new class of answerers. As LLMs and agents become the first responders to developer security questions, SecDevQA provides that instrument, and its category-to-artifact map (RQ1) extends this literature with a quantitative account of *which evidence* trustworthy answers rest on.

Positioning. Table V summarizes the comparison. The claim we make is precise: no existing benchmark combines (a) real security Q&A mined from developer forums, (b) external advisory artifacts as a context variable, (c) fact-grounded, condition-aware grading, and (d) systematic measurement of which artifact types are necessary for which question categories. We do not claim to be the first repository-level QA benchmark, nor the first security evaluation of LLMs.

TABLE V
COMPARISON WITH RELATED WORK. **SEC.** = SECURITY-SPECIFIC CORPUS; **ADV.** = EXTERNAL ADVISORY ARTIFACTS AS CONTEXT; **GRD.** = FACT-GROUNDED GRADING; **ABL.** = ARTIFACT-TYPE ABLATION. ✓ FULL; P PARTIAL; × NONE.

Paper	Sec.	Adv.	Grd.	Abl.
SWE-QA [3]	×	×	P	×
StackRepoQA [5]	×	×	×	×
CoReQA [4]	×	×	P	×
SecVulEval / CodeSecEval	✓	×	✓	×
SecureAgentBench	✓	×	✓	×
SecQA	✓	×	✓	×
SecDevQA (ours)	✓	✓	✓	✓

REFERENCES

[1] Y. Acar, M. Backes, S. Fahl, D. Kim, M. L. Mazurek, and C. Stransky, "You get where you're looking for: The impact of information sources on code security," in *2016 IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 289–305.

[2] F. Fischer, K. Böttinger, H. Xiao, C. Stransky, Y. Acar, M. Backes, and S. Fahl, "Stack overflow considered harmful? the impact of copy&paste on android application security," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 121–136.

[3] "Swe-qa: Can language models answer repository-level code questions?" Preprint, arXiv:2509.14635, 2025.

[4] "Coreqa: Uncovering potentials of language models in code repository question answering," Preprint, arXiv:2501.03447, 2025.

[5] "Beyond code snippets: Benchmarking llms on repository-level question answering," Preprint, arXiv:2603.26567, 2025.

[6] "Secvuleval: Benchmarking llms for real-world c/c++ vulnerability detection," Preprint, arXiv:2505.19828, 2025.

[7] "Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseceval," Preprint, arXiv:2407.02395, 2024.

[8] "Secureagentbench: Benchmarking secure code generation under realistic vulnerability scenarios," Preprint, arXiv:2509.22097, 2025.

[9] Z. Liu, "Secqa: A concise question-answering dataset for evaluating large language models in computer security," Preprint, arXiv:2312.15838, 2023.

[10] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE transactions on dependable and secure computing*, vol. 1, no. 1, pp. 11–33, 2004.

[11] A. Shostack, *Threat modeling: Designing for security*. John wiley & sons, 2014.

[12] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 2511–2522.

[14] S. Ye, D. Kim, S. Kim, H. Hwang, S. Kim, Y. Jo, J. Thorne, J. Kim, and M. Seo, "FLASK: Fine-grained language model evaluation based on alignment skill sets," in *International Conference on Learning Representations (ICLR)*, 2024.

[15] S. Kim, J. Suk, S. Longpre, B. Y. Lin, J. Shin, S. Welleck, G. Neubig, M. Lee, K. Lee, and M. Seo, "Prometheus 2: An open source language model specialized in evaluating other language models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[16] S. Kim, J. Suk, J. Y. Cho, S. Longpre, C. Kim, D. Yoon *et al.*, "The BiGGen Bench: A principled benchmark for fine-grained evaluation of language models with language models," in *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[17] R. K. Arora, J. Wei, R. Soskin Hicks, P. Bowman *et al.*, "HealthBench: Evaluating large language models towards improved human health," Preprint, arXiv:2505.08775, 2025.